EXPERIMENT - 6

ERROR DETECTION & ERROR CORRECTION.

AIM:

write a programme to implement error detection and correction using hamming code concept make a test run to input data stream and verify error correction feature.

Sender Program.

* Input is sender file should be a text of any length Program should convert the text to bi.

* Apply hamming code conept on the binary data and add redundant bits to it.

* Save in a file called channel.

Code (Sender.py)

```
data = list (map (int, input("Enter data bits: "), strip()))
m = len (data)
r = 0
while (2**r) < (m+r+1)
        r+=1
n = m+r
hamming = [0]*(n+1)
j = 0
for (i in range (0, n+1):
    if (1 * (i-1)) != 0:
        hamming [i] = data [j]
        j+=1
```

```
for p in range(r):
    pos = 2**p
    parity = 0
for i in range(1, r+1):
    if i in pos:
        parity ^= hamming[i]
hamming[pos] = parity
code = hamming(1)
print("Data bits:", data)
print("Hamming Code:", code)
```

Output:

Enter data bits: 1011

Data bits: [1,0,1,1]

Hamming code: [0,1,1,0,0,1,1]

Receiver Code (receiver.py)

```
received = list(map(int, input("Enter received hamming code:").strip()))
n = len(received)
r = 0
while (2**r) < n:
    r += 1
error pos = 0
for p in range(r):
    pos = 2**p
    parity = 0
```

```
for i in range(1, n+1):
    if i & pos:
        parity ^= received[i-1]
    if parity != 0:
        error_pos += pos
if error_pos == 0:
    print("No error detected")
else:
    print("Error detected at position:", error_pos)
    received[error_pos-1] ^= 1
    print("Corrected hamming code:", received_data[])
for i in range(1, n+1):
    if (i & (i-1)) != 0:
        data.append(received[i-1])
print("Original data list:", data)
```

Output:

Suppose hc: 0110011 change to 0111011

error detected at position: 4

Corrected hamming code: [0,1,1,0,0,1,1]

original data bits: [1,0,1,1]

Result:

Thus, the hamming code algorithm was successfully implemented in python. the sender generated the hamming code from the given data bits.